

SwasthyaSync

Privacy-First Personal Health Operating System
Detailed Project Report

Team Member	Team Member	Responsibilities
Mukul	System Architect	Full system architecture, backend development, database design, authentication & security, deployment & maintenance
Ashish	UI/UX Designer	User interface design, component styling, animations, responsive layouts, glassmorphism effects, user experience optimization
Punit	Project Manager	Project planning, task allocation, team coordination, requirement analysis, workflow optimization
Dikshant	Developer	Frontend development, API integration, state management, React components, feature implementation
Vaibhav	Project Tester	Testing & quality assurance, documentation, bug reporting, feature feedback, user support

1. Introduction

SwasthyaSync is a privacy-first, all-in-one personal health operating system — not just another health app, but a living ecosystem where individuals can track, store, analyse, and share every dimension of their health. From daily vitals and hydration to medical documents, doctor visits, nutrition, and beyond — all synced, and all under the user's control.

The system is built on a High-Maturity Enterprise Architecture following Hexagonal (Ports & Adapters) principles, Domain-Driven Design (DDD), and strict SOLID adherence. An internal EventBus decouples primary user actions from secondary side-effects (background jobs via BullMQ, analytics), ensuring a fast and resilient core pipeline.

The name Swasthya (स्वास्थ्य) is Sanskrit for 'health' — reflecting the project's India-first identity and its ambition to serve users with culturally-aware health intelligence unavailable in Western-centric applications.

2. Project Objectives

- Build a scalable, privacy-first health OS with Hexagonal Architecture for maximum testability and modularity.
- Implement secure session-based authentication using Better Auth with email/password and Google OAuth.
- Follow SOLID principles and clean Domain-Driven Design for long-term maintainability and extensibility.
- Create an intelligent Fitness Engine with adaptive TDEE calculation (28-day sliding window, 7-day SMA smoothing).
- Provide a Unified Pulse Dashboard that aggregates data from multiple repositories in parallel.
- Enable a Health Vault for document management (prescriptions, lab reports, X-rays) with tag-based search.
- Track holistic wellness: sleep quality, intermittent fasting, hydration (multi-container), and nutrition macros.
- Decouple primary operations from secondary side-effects using an internal EventBus and BullMQ job queue.
- Deploy to Vercel with PostgreSQL (Supabase) and Redis for production-grade background processing.
- Lay the foundation for future AI health insights, wearable integration, and a Doctor Portal with temporary access links.

3. Tech Stack

Layer	Technology	Version	Purpose
Frontend Framework	Next.js (App Router)	15 / 16	SSR, file-based routing, API routes
UI Library	React	19	Component-based UI with hooks & context
Language	TypeScript (strict)	5.7	Static typing, generics, strict null checks
API Layer	tRPC	11	End-to-end type-safe API without REST boilerplate

Layer	Technology	Version	Purpose
Styling	Tailwind CSS	4	Utility-first CSS with dark-mode support
Icons	Lucide React + React Icons	Latest	Comprehensive SVG icon libraries
Database	PostgreSQL + Drizzle ORM	0.45	Type-safe SQL ORM with migration tooling
Authentication	Better Auth	1.6	Email/password + Google OAuth, session mgmt
Background Jobs	BullMQ + Redis	5.x	Reliable job queues for async processing
Validation	Zod	4.x	Schema validation at all API boundaries
Date Utilities	date-fns	4.x	Date formatting, SMA windows, interval ops
Logging	Pino	10.x	Structured JSON logging
Server State	TanStack Query v5	5.x	Server-state caching via tRPC integration
Deployment	Vercel + Supabase	—	Serverless hosting & managed PostgreSQL

4. System Architecture

SwasthyaSync follows a strict Hexagonal (Ports & Adapters) Architecture combined with Domain-Driven Design (DDD), organising the codebase into four clearly separated layers per feature module. Each module (health, fitness, medical) is a self-contained hexagon with its own domain, application, and infrastructure sub-directories, wired together at a single Composition Root. The tRPC API layer sits outside all module boundaries and depends only on the Composition Root's exported API.

4.1 Architecture Layers

Layer	Responsibility & Rules
Domain Layer (/domain)	Technology-agnostic core. Contains pure TypeScript interfaces (Entities), repository Port contracts, and Domain Events (e.g., MetricLoggedEvent). No framework, database, or HTTP code touches this layer.
Application Layer (/application)	Orchestrates business logic through atomic Use Cases (LogMetricUseCase, GetUnifiedPulseDashboardUseCase, etc.). Use Cases depend only on repository interfaces. DTOs cross the Application boundary; Mappers act as Anti-Corruption Layers. Zod validators guard all entry points.
Infrastructure Layer (/infrastructure)	The 'Adapters' connecting the domain to the outside world. Contains Drizzle ORM implementations, BullMQ job queue adapters, Redis client, Open Food Facts API integration, and Exercise API service. All classes implement Port interfaces from the Domain layer.
Composition Root (*composition.ts)	The single wiring point per module. Instantiates concrete infrastructure classes, injects them into Use Cases, and subscribes Infrastructure side-effects to Domain Events via the EventBus. The rest of the application interacts only with the Composition Root's exported API.
API / Presentation (tRPC Routers + React)	Next.js App Router pages and layouts form the UI. tRPC routers act as the gateway, handling Zod input validation, authentication (protectedProcedure), and structured error formatting with SuperJSON serialisation.

4.2 Event-Driven Decoupling

Side-effects are handled asynchronously through an internal EventBus (a typed wrapper around Node.js EventEmitter). When a user logs a metric, LogMetricUseCase saves the record and publishes a health.metric_logged event. The Composition Root subscribes to this event and enqueues a BullMQ background job (process-metric) for AI analysis — completely decoupled from the primary request path. The user receives an immediate response while heavy processing happens asynchronously in the Redis-backed queue.

4.3 Project Structure

Path	Contents
src/app/	Next.js App Router pages and layouts (UI Layer)
src/components/	Shared UI components (Navbar, Sidebar, landing sections)
src/lib/	Shared utilities: DB client (Drizzle), EventBus, Better Auth config, env validation, logger, Redis, BullMQ queue

Path	Contents
src/modules/health/	Health module: domain entities, MetricRepository port, LogMetricUseCase, GetUnifiedPulseDashboardUseCase, DrizzleMetricRepository, health.composition.ts
src/modules/fitness/	Fitness module: Workout, Nutrition, FoodLog, Goal, Metabolic domains; TdeeCalculator, CalorieBurnEstimator; 4 Drizzle repositories; fitness.composition.ts
src/modules/auth/	Auth infrastructure: Better Auth schema and DB adapter
src/server/	tRPC routers (healthRouter, fitnessRouter, appRouter) and createTRPCContext
src/types/ & src/utils/	Type declarations and tRPC client utility
drizzle/	SQL migration files generated by Drizzle Kit

5. SOLID Principles Implementation

5.1 Single Responsibility Principle (SRP)

- **LogMetricUseCase**: responsible only for saving a metric and emitting a domain event.
- **DrizzleMetricRepository**: responsible only for PostgreSQL CRUD on the health_metrics table.
- **TdeeCalculator**: responsible only for computing adaptive TDEE from weight and calorie data.
- **CalorieBurnEstimator**: responsible only for MET-based calorie burn estimation with age/gender adjustments.
- **EventBus**: responsible only for typed publish/subscribe event dispatch.
- **MetricMapper**: responsible only for translating HealthMetric entities to MetricDTOs.

5.2 Open/Closed Principle (OCP)

- **Repository interfaces**: define stable contracts. New storage backends can be added by implementing the interface without changing any use case or domain code.
- **fitnessEngine Composition Root**: new use cases are added by wiring them here only — existing tRPC routes and use cases are untouched.

5.3 Liskov Substitution Principle (LSP)

- **DrizzleFoodLogRepository, DrizzleWorkoutRepository, DrizzleGoalRepository**: fully satisfy their respective Port interfaces and can be substituted with mock implementations in unit tests without breaking use case behaviour.

5.4 Interface Segregation Principle (ISP)

- **MetricRepository**: exposes only save() and findById() — not a bloated generic CRUD interface.
- **FoodLogRepository**: exposes only save, findByDate, getTodayCalories, delete — purpose-driven.
- **WorkoutRepository**: similarly focused on workout-specific queries.

5.5 Dependency Inversion Principle (DIP)

- **Use Cases**: receive repository interfaces via constructor injection; they have zero knowledge of PostgreSQL, Drizzle, or any concrete class.
 - **tRPC Routers**: depend on the Composition Root's exported API, not on use cases or repositories directly.
 - **BullMQ / EventBus**: job enqueueing is subscribed in the Composition Root, keeping Domain and Application layers free of infrastructure imports.
-

6. Design Patterns

Pattern	Implementation
Hexagonal / Ports & Adapters	Each module defines Port interfaces in /domain/repositories. Infrastructure adapters implement these ports. The domain never imports from infrastructure, making all business logic fully testable in isolation.
Repository Pattern	DrizzleMetricRepository, DrizzleFoodLogRepository, DrizzleWorkoutRepository, and DrizzleGoalRepository abstract all database operations behind typed interfaces defined in the domain layer.
Use Case Pattern	Each atomic business action is a dedicated class (LogMetricUseCase, LogFoodUseCase, GetUnifiedPulseDashboardUseCase, UpdateAdaptiveTdeeUseCase, etc.) with a single execute() method, keeping business logic focused and testable.
Composition Root	health.composition.ts and fitness.composition.ts are the sole wiring points — infrastructure instantiation, constructor injection, and EventBus subscription all happen exclusively here.
Observer / EventBus	The EventBus (typed Node.js EventEmitter wrapper) allows domain events (MetricLoggedEvent) to trigger infrastructure side-effects (BullMQ job enqueue) without coupling Domain and Infrastructure layers.
Data Mapper / ACL	MetricMapper.toDTO() and equivalent mappers act as Anti-Corruption Layers, translating between internal Domain entities and externally-visible DTOs to prevent domain model leakage.
Strategy Pattern	TdeeCalculator (adaptive energy balance) and CalorieBurnEstimator (MET-based, Brzycki 1RM) encapsulate interchangeable algorithms that can be swapped without affecting callers.
Facade Pattern	fitnessEngine acts as a high-level facade exposing a unified API (nutrition.logFood, workoutTracking.logWorkout, getDailySummary) to tRPC routers, hiding internal complexity.
Protected Procedure (Middleware)	tRPC's protectedProcedure middleware enforces authentication on all routes by validating the Better Auth session before delegating to the use case.
Provider Pattern	TRPCProvider wraps the Next.js app with TanStack Query context for seamless server-state management and cache invalidation on the client.

7. Database Design

SwasthyaSync uses PostgreSQL (hosted on Supabase) with Drizzle ORM for type-safe schema definition and migrations. The schema is organised into three domains spanning 21 tables.

7.1 Domain Overview

Domain	Tables	Purpose
Auth	user, session, account, verification	User identity, OAuth providers, sessions, email verification
Health	health_metrics, medical_appointments, health_vault_records	Core vitals tracking, doctor appointments, document storage
Fitness	14 tables (fitness_*)	Workouts, nutrition, TDEE, sleep, fasting, hydration, goals, AI insights, family access, external providers

7.2 Key Tables & Schema

Table	Key Fields	Notes
user	id (PK, text), name, email (unique), emailVerified, image, createdAt	Central entity; all user-owned tables FK to user.id
session	id, token (unique), expiresAt, userId (FK), ipAddress, userAgent	Better Auth session management; HTTP-only cookie token
health_metrics	id (PK, integer, auto-identity), userId (FK), type, value (text), unit, timestamp	Polymorphic value column stores numeric vitals and string blood groups
medical_appointments	id, userId (FK), doctorName, specialty, appointmentDate, location, notes, status	Status: scheduled / completed / cancelled
health_vault_records	id, userId (FK), title, category, fileUrl, recordDate, tags, createdAt	Supports Lab Report, Prescription, X-Ray, Dental, Insurance, etc.
fitness_workout_logs	id, userId (FK), workoutId (FK), date, durationMinutes, caloriesBurned, sets (jsonb)	Sets stored as JSONB array for flexible exercise data
fitness_nutrition_logs	id, userId (FK), date, mealType, foodName, quantity, unit, nutrients (jsonb)	Nutrients snapshotted at log time for historical accuracy
fitness_daily_summaries	id, userId (FK), date, totalCaloriesIn, totalCaloriesOut, netCalories, weightKg, waterMI	Aggregated daily view for the Unified Pulse dashboard
fitness_tdee_logs	id, userId (FK), date, tdee, confidence (HIGH/MEDIUM/LOW), isFallback, calculationContext (jsonb)	Tracks adaptive TDEE history with confidence scoring

Table	Key Fields	Notes
fitness_sleep_logs	id, userId (FK), date, bedTime, wakeTime, totalDurationMinutes, qualityScore, stages (jsonb)	Supports sleep stage breakdown (REM, Light, Deep)
fitness_goals	id, userId (FK), type, goalDate, targetNutrients (jsonb), targetValues (jsonb), status	Supports lose_weight, build_muscle, maintain goal types
fitness_family_access	id, ownerId (FK→user), memberId (FK→user), permissions (jsonb), isActive	Two FKs to user — enables caregiver / family sharing mode
fitness_ai_insights	id, userId (FK), type, content, source, createdAt	Stores AI-generated health insights for display

7.3 Entity Relationships

- A User has many Sessions and Accounts (one-to-many via userId FK).
- A User logs many HealthMetrics, one per measurement event (one-to-many).
- A User schedules many MedicalAppointments and stores many HealthVaultRecords.
- A User creates Fitness Workout Templates and logs Workout Sessions referencing those templates.
- A User logs Nutrition entries, Sleep logs, Water intake, and Fasting windows daily.
- A User receives AI Insights and connects External Providers (Garmin, Fitbit, etc.).
- FamilyAccess links two Users (owner ↔ member) with a JSONB permissions object for granular role control.
- DailySummaries and TdeeLogs aggregate multi-source data for the Unified Pulse dashboard.

8. Feature Modules

Module	Description
Health Tracking	Tracks core vitals — Height, Weight, Blood Group, BMI (auto-calculated from height/weight). Data flows through the full hexagonal pipeline: tRPC mutation → Zod validator → LogMetricUseCase → DrizzleMetricRepository → EventBus → BullMQ job. GetMetricsUseCase returns the latest value per metric type; GetHealthInsightsUseCase generates trend insights from historical data.
Fitness & Workout	Supports detailed workout logging (sets, reps, RPE, workout templates with exercise IDs) and a simpler quick-log flow (exercise name, sets, reps, duration). CalorieBurnEstimator uses MET values with category/level lookup, gender adjustment (10% reduction for female), age factor (0.5% per 5 years over 30), and the Brzycki 1RM formula for intensity-aware strength estimation.
Nutrition & Calorie Tracking	Integrates with the Open Food Facts API for food search (no API key required). LogFoodUseCase persists entries with snapshotted macro data. GetFoodLogsByDateUseCase returns entries grouped by meal type (breakfast, lunch, dinner, snacks). GetTodayCaloriesUseCase sums the day's intake for dashboard display.
Metabolic Engine (Adaptive TDEE)	TdeeCalculator implements a scientific 28-day sliding window analysis: (1) linear interpolation fills missing weight measurements, (2) 7-day Simple Moving Average smooths weight trends to remove noise, (3) TDEE = avg_daily_intake – (daily_weight_change_kg × 7700 kcal). Confidence levels (HIGH / MEDIUM / LOW) are assigned based on data density. Results are persisted in fitness_tdee_logs.
Medical Records — Health Vault	Document management for prescriptions, lab reports, X-rays, dental, eye checkups, insurance, and more. Tag-based search and category filtering with category-specific icons. Doctor appointment scheduling with specialty, location, date, and notes. DrizzleMedicalRepository provides the infrastructure adapter for both sub-features.
Unified Pulse Dashboard	GetUnifiedPulseDashboardUseCase runs parallel repository queries — pulling vitals, fitness summaries, and medical data simultaneously — and aggregates them into a single dashboard payload. This 'master heartbeat' of the user's health is the primary view after login.
Holistic Wellness	Sleep logs with quality scoring (1–100) and optional stage breakdown (REM/Light/Deep). Intermittent fasting timer with start/end tracking and completion status. Multi-container hydration tracking (custom bottle sizes, water intake history, daily progress toward 3L goal).
Goals & Progress	SetSimpleGoalUseCase records user goals (lose_weight, build_muscle, maintain). GetGoalWithProgressUseCase cross-references goals with food logs and workout history to compute real-time progress, powering the Goals tab in the Fitness page.

9. Authentication & Security

9.1 Authentication Flow

1. User submits email/password via the Login or Sign Up form in the Next.js App Router page.
2. Better Auth validates credentials, hashes passwords (bcrypt internally), and creates a session record in the PostgreSQL database.
3. A secure, HTTP-only session cookie is set on the response via Better Auth's `nextCookies()` plugin.
4. Next.js middleware intercepts protected routes and validates the session on every request before rendering.
5. The tRPC `protectedProcedure` middleware calls `auth.api.getSession()` and throws `UNAUTHORIZED` if the session is absent or expired.
6. Google OAuth is conditionally enabled via `socialProviders` config only when `AUTH_GOOGLE_ID` and `AUTH_GOOGLE_SECRET` environment variables are present.
7. Session data (user id, email, name) is available in tRPC context (`ctx.session.user`) for all protected procedures.

9.2 Security Measures

- **HTTP-Only Session Cookies:** Session tokens are stored in HTTP-only cookies, preventing JavaScript access and XSS-based token theft.
 - **Password Hashing:** Better Auth uses bcrypt internally for all password storage — plain-text passwords never touch the database.
 - **Environment-Gated OAuth:** Google OAuth is conditionally enabled only when credentials are present, preventing misconfiguration errors in deployments.
 - **Trusted Proxy Headers:** The `trustedProxyHeaders` flag ensures correct IP detection behind Vercel's edge network proxies.
 - **Zod Input Validation:** All tRPC mutations and queries validate input against Zod schemas before reaching any use case or repository.
 - **Session-Scoped Data Access:** Every repository query filters by `ctx.session.user.id` — users can only read and write their own data.
 - **Strict TypeScript:** TypeScript strict mode eliminates null/undefined runtime errors and enforces type contracts at compile time.
 - **Environment Validation:** `src/lib/env.ts` validates all required environment variables at startup using a Zod schema, failing fast on misconfiguration.
-

10. API Design (tRPC)

SwasthyaSync replaces traditional REST APIs with tRPC v11, providing end-to-end type safety between the Next.js server and React client without code generation or schema files. All routes are split into healthRouter and fitnessRouter, combined in a single appRouter mounted at `/api/trpc/[trpc]`.

10.1 Health Router

Procedure	Type	Description
health.getMetrics	Query	Returns the latest value per metric type for the current user
health.getTodayHydration	Query	Returns today's total hydration in ml
health.getHealthInsights	Query	Returns trend-based health insights from historical metrics
health.getUnifiedPulse	Query	Master dashboard — aggregates vitals, fitness, and medical data in parallel
health.logMetric	Mutation	Saves a new health metric (weight, height, blood group, hydration); emits domain event to trigger background job

10.2 Fitness Router (Selected Endpoints)

Procedure	Type	Description
fitness.searchFood	Query	Searches Open Food Facts API; returns array of {foodName, calories}
fitness.logFood	Mutation	Logs a food entry with macro snapshot for a specific meal type and date
fitness.getFoodLogsByDate	Query	Returns food logs grouped by meal type for a given date
fitness.getTodayCalories	Query	Returns total calories consumed today
fitness.deleteFoodLog	Mutation	Deletes a food log entry (session-scoped security check)
fitness.logSimpleWorkout	Mutation	Logs a workout (exercise name, sets, reps, duration, date)
fitness.getSimpleWorkoutsByDate	Query	Returns all workouts for a given date
fitness.getTodayCaloriesBurned	Query	Returns total calories burned via workouts today
fitness.getDailySummary	Query	Returns combined calorie in/out summary for a given date
fitness.setSimpleGoal	Mutation	Sets user fitness goal (lose_weight / build_muscle / maintain)

Procedure	Type	Description
fitness.getGoalProgress	Query	Returns goal details with real-time progress computation
fitness.syncTdee	Mutation	Triggers adaptive TDEE recalculation for the current user
fitness.logSleep	Mutation	Logs a sleep session with quality score and optional stage breakdown
fitness.logWater	Mutation	Logs water intake in ml with optional container reference
fitness.startFast	Mutation	Starts an intermittent fasting window with target duration in hours

11. Frontend Design

11.1 Page Structure

Page	Description
Landing Page (/)	Marketing-style page with Hero, Problem, ProblemSolved, and Solution sections. Brand-orange (#FF4A20) identity, feature icons, and a card-based responsive grid layout with a CTA: 'Build Your Own Health Vault'.
Login (/login)	Authentication form with email/password fields, error handling, and automatic redirection post-login. Supports Google OAuth as an alternative sign-in method.
Sign Up (/signup)	Registration form with name, email, and password fields with validation and informational messaging.
Dashboard (/dashboard)	Personalised greeting with date. Health Metric cards (Height, Weight, Blood Group, BMI auto-calc). Upcoming Doctor Visits widget. Hydration Summary with daily progress bar. Health Vault CTA. Tip of the Day. Data sourced via tRPC queries to the Unified Pulse use case.
Fitness (/fitness)	Multi-tab interface: Nutrition (food search, meal logging, macro breakdown, calorie goal), Workout (exercise logging, calories burned), Goals (progress tracking), and Daily Summary view.
Hydration (/hydration)	Multi-beverage tracking with custom ml input. Period-based views (Daily, Weekly, Monthly, Quarterly, Yearly).
Health Vault (/health-vault)	Document management with category-based icons, tag-based search and filtering, and date/doctor/notes fields per record.
Doctor Meetup (/doctor-meetup)	Appointment scheduling and tracking with specialty-based filtering and notes. Upcoming visits surfaced on the Dashboard.

11.2 Shared Components

Component	Description
Navbar	Top navigation with user profile display, logout dropdown, and responsive hamburger menu.
Sidebar	Responsive navigation linking all main pages; collapses to horizontal on mobile. Protected route wrapper lives in the layout.
TRPCProvider	Wraps the app with TanStack Query and tRPC client context; uses SuperJSON for correct Date serialisation.
Landing Sections	Hero, Problem, ProblemSolved, Solution, and SectionHeader are modular, independently styled landing page components.
Protected Layout	/app/(app)/layout.tsx enforces authentication and renders Sidebar + Navbar as the persistent app shell for all inner pages.

11.3 Design Philosophy

Aspect	Detail
Brand Color	#FF4A20 (energetic, health-forward orange-red)
Typography	Calibri / Inter / Poppins — modern, readable, optimised for data-dense UIs
Aesthetic	Clean, card-based layouts — 'Apple Health meets Indian cultural warmth'
Mobile-First	Sidebar collapses, grids reflow; all touch targets sized for mobile interaction
Dark Mode	Tailwind CSS v4 dark mode with system preference detection and localStorage persistence

12. OOP & Design Concepts

Concept	Implementation in SwasthyaSync
Abstraction	Repository interfaces (MetricRepository, FoodLogRepository, etc.) hide all database details. Use Cases never see SQL — only clean method calls. The DomainEvent interface abstracts the event contract across all modules.
Encapsulation	Each Use Case encapsulates a complete business operation. The EventBus encapsulates the emitter. Composition Root files encapsulate all wiring logic, exposing only a clean exported API to tRPC routers.
Dependency Injection	All Use Cases receive their dependencies (repositories, EventBus) via constructor parameters. The Composition Root is the sole location where concrete classes are instantiated — no service locators.
Polymorphism	Any class implementing MetricRepository can substitute the Drizzle implementation (e.g., an in-memory mock for unit tests). TdeeCalculator and CalorieBurnEstimator algorithms can be swapped by providing alternative implementations.
Generics & Type Safety	TypeScript strict generics are used throughout: repository interfaces typed per entity, tRPC procedures carry full input/output types end-to-end, and Zod schemas provide runtime type narrowing at API boundaries.
Value Objects (Domain Logic)	TdeeResult, ExerciseSession, UserStats, and TdeeConfidence are typed interfaces acting as immutable Value Objects — self-describing data carriers used exclusively by domain logic classes.
Event-Driven Design	DomainEvent interface + EventBus implements the Observer pattern for async decoupling. MetricLoggedEvent carries a strongly-typed payload (HealthMetric) that subscribers consume safely without casting.

13. SDLC Methodology

The project follows an Agile / iterative development methodology with continuous architectural evolution documented in VISION.md and context.md. The system matured from a Vite + React prototype to a full enterprise-grade Next.js Hexagonal Architecture across multiple well-defined phases.

Phase	Description
Phase 1 Foundation	Defined core requirements: health tracking, authentication, hydration, document vault, and doctor visits. Migrated from Vite + react-router to Next.js App Router. Set up PostgreSQL with Drizzle ORM and Better Auth.
Phase 2 Architecture Hardening	Introduced Hexagonal Architecture with Domain, Application, and Infrastructure layers. Created Composition Roots per module. Added EventBus for decoupling side-effects from primary operations.
Phase 3 Fitness Engine	Built the full fitness module: workout logging, food search (Open Food Facts API), nutrition tracking with macro snapshots, adaptive TDEE engine (TdeeCalculator), and calorie burn estimation (CalorieBurnEstimator).
Phase 4 Holistic & Goals	Added sleep logging (with quality score and stage breakdown), intermittent fasting, multi-container hydration, and goal tracking with real-time progress via GetGoalWithProgressUseCase.
Phase 5 tRPC & Unified Pulse	Migrated API layer from Next.js route handlers to tRPC v11 for end-to-end type safety. Built GetUnifiedPulseDashboardUseCase to aggregate multi-source data in parallel for the master dashboard.
Roadmap (Phases 6–10)	Phase 6: Indian-first calorie database (regional cuisines, Thali Builder). Phase 7: Prescription OCR & Medicine Reminders. Phase 8: Family Health Hub & Caregiver Mode. Phase 9: Doctor Portal with Temporary Access Links. Phase 10: AI Health Intelligence (trend prediction, anomaly detection, health score, ABDM/ABHA integration).

14. Conclusion

SwasthyaSync demonstrates a professionally architected, full-stack health platform that strictly adheres to SOLID principles, Hexagonal Architecture, Domain-Driven Design, and event-driven patterns. The Ports & Adapters structure ensures that no business logic is contaminated by infrastructure concerns — the Domain and Application layers are fully technology-agnostic and testable in isolation.

The tRPC API layer delivers end-to-end type safety without REST boilerplate, while BullMQ and the internal EventBus enable fast user-facing responses with heavy processing deferred asynchronously. The adaptive TDEE engine, MET-based calorie estimator, Brzycki 1RM formula, and parallel Unified Pulse aggregation represent genuine algorithmic depth beyond standard CRUD applications.

With a clear 10-phase roadmap covering Indian-first nutrition intelligence, Prescription OCR, Family Health Hub, AI trend prediction, and ABDM/ABHA integration, SwasthyaSync is positioned not merely as a capstone project but as a viable foundation for a production health revolution — one sync at a time.

“We’re not building another health app. We’re building a health revolution — one sync at a time.”